

Verantwoordingsdocument — WebBazar

Verantwoordingsdocument — WebBazar.....	1
Inleiding.....	2
Technische keuzes en motivatie.....	3
1. Bewuste inzet van een conceptuele prototype-architectuur.....	3
4. Backend beveiliging via role-based autorisatie in SecurityConfig én method-level security.....	4
5. Validatie en input-sanering via DTO's (backend).....	5
6. Gelaagde backend architectuur met duidelijke scheiding van verantwoordelijkheden.....	6
7. Realistische server-side bestandsdownload logica (met huurcontrole).....	7
8. Gebruik van schema.sql en data.sql voor deterministische initiële data.....	8
9. Fallback-mechanisme bij het checkout-proces (frontend).....	8
10. Globale exception handling voor consistente fout responses (backend).....	9
11. Domain-level autorisatie met OwnershipGuard en @PreAuthorize (backend)....	10
Reflectie op het leerproces.....	11
Limitaties en mogelijke doorontwikkeling.....	12
1. Geen integratie met een echte betaalmethode.....	12
2. Onvolledige testdekking.....	12
3. Alleen lokale bestandsopslag.....	13
4. Geen e-mailnotificaties en wachtwoord reset proces.....	13
5. Huurverval nog niet volledig geautomatiseerd en gesignaleerd.....	14
Conclusie.....	15
Bronvermelding.....	16
GitHub-links.....	16

Inleiding

In dit verantwoordingsdocument beschrijf ik de technische keuzes en overwegingen die ik heb gemaakt bij de ontwikkeling van WebBazar: een full-stack boekenplatform waarin gebruikers boeken kunnen kopen of huren, bestellingen kunnen beheren en digitale bestanden kunnen downloaden. De backend van de applicatie is gebouwd met Spring Boot, terwijl de frontend is ontwikkeld in React. Beide componenten communiceren via een REST API die is beveiligd met JWT-authenticatie.

Dit document richt zich niet alleen op wat ik heb gebouwd, maar vooral op waarom ik bepaalde implementatiekeuzes heb gemaakt en wat ik daarbij heb geleerd. Ik bespreek de keuzes rondom architectuur, beveiliging, datamodellering, state-management, foutafhandeling en testaanpak. Daarnaast reflecteer ik op de praktische uitdagingen die ik tijdens het ontwikkelproces tegenkwam en hoe ik deze heb opgelost.

De kernfunctionaliteiten van WebBazar omvatten onder andere registratie en inloggen, rol-gebaseerde autorisatie, beheer van producten door een administrator, bestellen en huren van boeken en het downloaden van digitale boekbestanden (zoals PDF-e-books) en facturen. Deze functionaliteiten zijn daadwerkelijk geïmplementeerd en vormen de basis voor de analyse en reflectie in de komende hoofdstukken.

Technische keuzes en motivatie

1. Bewuste inzet van een conceptuele prototype-architectuur

Omdat dit project binnen het kader van een eindopdracht valt en niet als productiesoftware is ontwikkeld, heb ik gekozen voor een architectuur die realistische praktijkstructuren simuleert, maar niet op alle punten volledig productierijp hoeft te zijn (bijvoorbeeld het ontbreken van een echte betalingsprovider en cloud bestandsopslag. De focus lag op inzicht in technologie en architectuur, niet op commerciële exploitatie.

Reflectie:

Deze keuze gaf mij de ruimte om mij te richten op de kwaliteit van de code, de architectuur en de beveiliging, in plaats van op randzaken zoals compliancy met echte betaalstromen. Tegelijkertijd heb ik gemerkt dat je zelfs bij een prototype al snel tegen “productieproblemen” aanloopt, zoals foutafhandeling, dataconsistentie en robuuste downloadlogica.

2. Structuur van API-aanroepen via gecentraliseerde Axios-helpers (frontend)

Aan de frontend heb ik een eigen Axios-instance geconfigureerd waarin de base URL en de JWT-header automatisch worden toegevoegd. Dit voorkomt herhaalde configuratie van headers in elke component en maakt debugging efficiënter. Voor sommige schermen (zoals orderdetail) gebruik ik een extra Axios-instance met een response-interceptor die bij een 401-response automatisch het opgeslagen token opruimt.

De hoofdlijn is dat API-configuratie (baseURL, headers) centraal is georganiseerd in plaats van verspreid over alle componenten.

Reflectie:

In een eerdere versie bouwde ik elke API-call handmatig op in losse components, wat leidde tot duplicated logic en onoverzichtelijke foutafhandeling. Door centrale Axios-services te introduceren heb ik de codebase beter kunnen structureren en meer inzicht gekregen in DRY-principes en het gebruik van interceptors. In een volgende iteratie wil ik alle API-calls volledig via één gedeelde instance laten lopen, zodat ook foutafhandeling (bijvoorbeeld bij 401) overal identiek verloopt.

3. Gebruik van Context API voor gebruikers- en winkelmand-state (frontend)

Voor state management heb ik gekozen voor de React Context API in combinatie met hooks, in plaats van Redux. De omvang en complexiteit van de applicatie vragen niet om een volledige Redux-architectuur, terwijl Context API voldoende mogelijkheden biedt om gedeelde state (zoals de ingelogde gebruiker, het JWT-token en de winkelmand) centraal op te slaan.

Reflectie:

Aan het begin had ik last van prop-drilling: componenten gaven veel props door naar onderliggende componenten, wat de leesbaarheid verminderde. Door AuthContext en CartContext in te voeren, heb ik geleerd om state op een hoger niveau te beheren en sessies persistent te maken via localStorage, zodat een gebruiker ingelogd blijft na het verversen van de pagina. Dit heeft ook geholpen om private routes en rolgebaseerde weergave van de UI (bijvoorbeeld adminmenu's) overzichtelijk te implementeren.

4. Backend beveiliging via role-based autorisatie in SecurityConfig én method-level security

In de backend heb ik de basis van de toegangscontrole geïmplementeerd in de SecurityFilterChain binnen SecurityConfig. Hier worden de globale regels vastgelegd, zoals:

- GET /api/products/** → openbaar toegankelijk
- POST/PUT/DELETE /api/products/** → alleen toegankelijk voor ADMIN
- overige endpoints → alleen toegankelijk voor geauthenticeerde gebruikers

Voor fijnmazigere autorisatie maak ik aanvullend gebruik van method-level security met `@PreAuthorize`. Daarin gebruik ik een Ownership Guard-component die controleert of de ingelogde gebruiker eigenaar is van een bepaalde order of orderregel, bijvoorbeeld bij:

- GET `/api/orders/{id}/invoice`
- GET `/api/downloads/{orderId}`

Hiermee combineer ik globale rol regels in `SecurityConfig` met domeinspecifieke regels op methode niveau.

Reflectie:

In een eerdere iteratie vertrouwde ik te veel op de frontend voor het verbergen van admin-functionaliteit. Via tools als Postman werd duidelijk dat dit niet voldoende is. Ik heb hiervan geleerd dat autorisatie altijd op backend-niveau moet worden afgedwongen. De combinatie van centrale configuratie in `SecurityConfig` en specifieke `@PreAuthorize`-regels met een `OwnershipGuard` helpt om zowel overzicht te houden als heel gericht eigenaarschap af te dwingen.

5. Validatie en input-sanering via DTO's (backend)

Aan de backend zijde maak ik gebruik van DTO's om gebruikersinput te ontvangen en te valideren, in plaats van direct met entity-klassen te werken. DTO's zorgen ervoor dat database-modellen niet direct blootstaan aan gebruikersinput en maken het eenvoudiger om validatie-annotaties (`@Email`, `@NotBlank`, `@Size`, etc.) toe te passen. Controllers gebruiken `@Valid` om deze validatie automatisch te laten uitvoeren.

Reflectie:

In de beginfase probeerde ik validatie direct op de entiteiten en in controllers te doen. Dit leidde tot rommelige code en inconsistente foutmeldingen. Door validatie te verplaatsen naar DTO's en gebruik te maken van `@Valid` in de controllers, is de structuur helderder geworden en kon ik foutafhandeling beter uniform maken. Dit sluit bovendien goed aan bij het principe dat entiteiten domeinmodellen zijn, niet automatisch API-modellen.

6. Gelaagde backend architectuur met duidelijke scheiding van verantwoordelijkheden

De backend is opgezet volgens een klassieke gelaagde architectuur met afzonderlijke packages voor controllers, services, repositories, entities en DTO's.

- Controllers zijn verantwoordelijk voor HTTP-afhandeling en mapping van DTO's.
- Services bevatten de business logica (zoals orderverwerking, prijsberekening en huurconstructies).
- Repositories handelen de database-interacties af via Spring Data JPA.

Reflectie:

Deze indeling heeft mij geholpen om SOLID-principes toe te passen. Wanneer logica te veel in controllers terechtkwam, merkte ik direct dat testen en hergebruik lastig werden. Door actief logica naar de service-laag te verplaatsen, werd de code beter toetsbaar en herbruikbaar. De structuur maakt het ook eenvoudiger om later extra business regels toe te voegen zonder de controllers te vervuilen.

7. Realistische server-side bestandsdownload logica (met huurcontrole)

Downloads van PDF-facturen en e-books verlopen via beveiligde backend-endpoints. Daarbij wordt gecontroleerd of de ingelogde gebruiker de eigenaar is van de betreffende order of orderregel via de OwnershipGuard. Daarnaast is er server-side modellering van huurperioden via een aparte Rental-entiteit, waarin start- en einddatum van de huur worden vastgelegd.

Bij het downloaden van e-books wordt expliciet gecontroleerd of:

- de huur al is ingegaan ($\text{startdatum} \leq \text{nu}$)
- de huur nog niet is afgelopen ($\text{nu} \leq \text{einddatum}$)

Zo niet, dan retourneert de backend een passende HTTP-status (bijvoorbeeld LOCKED of GONE) en wordt de download geweigerd.

Verder wordt bij downloads defensief omgegaan met bestandsnamen en paden (pad-normalisatie en het opschonen van bestandsnamen) om risico's zoals path traversal en onveilige filenamen te beperken.

Reflectie:

Ik heb geleerd om toegang tot bestanden niet alleen te baseren op “ingelogd zijn”, maar expliciet te koppelen aan eigenaarschap en (in het datamodel) aan huurcondities. Ook heb ik inzicht gekregen in best practices rondom veilige bestandsdownloads. In een volgende iteratie wil ik deze huurcondities nog verder uitbreiden met bijvoorbeeld notificaties richting de gebruiker wanneer een huur bijna verloopt.

8. Gebruik van schema.sql en data.sql voor deterministische initiële data

In plaats van Hibernate volledig automatisch de database te laten genereren, gebruik ik schema.sql en data.sql om de database-structuur en testdata expliciet vast te leggen. Hiermee:

- voorkom ik onverwachte wijzigingen in het datamodel;
- documenteer ik de tabelstructuren en relaties expliciet;
- kan ik bij iedere applicatiestart dezelfde consistente basisdata inladen (inclusief testgebruikers en voorbeeldproducten).

Reflectie:

Door zelf de FK-constraints, indices en tabellen te definiëren, heb ik een beter begrip gekregen van relationeel databaseontwerp en migraties. Dit sluit beter aan bij hoe professionele projecten met tools als Flyway of Liquibase werken. Het feit dat de seed-data voorspelbaar is, helpt bovendien bij het schrijven en draaien van integratietests.

9. Fallback-mechanisme bij het checkout-proces (frontend)

Wanneer de backend tijdens checkout tijdelijk niet reageert of een fout retourneert, genereert de frontend zelf een concept-factuur (PDF) en een concept-XML-bestand op basis van de gegevens in het winkelmandje. Ook worden eventuele backend fouten gelogd in de console en wordt het winkelmandje na deze stap geleegd, zodat de gebruiker niet met “halve” bestellingen blijft zitten. In de normale “happy flow” wordt de order opgeslagen in de backend, wordt de factuurserver-side gegenereerd en worden e-books via beveiligde endpoints gedownload.

Reflectie:

Aanvankelijk ging ik uit van de “happy flow” waarin de backend altijd goed reageert. In de praktijk bleek dat netwerkproblemen of serverfouten realistische scenario's zijn. Door een fallback in de frontend te bouwen, heb ik geleerd om robuuster te denken over foutscenario's en de gebruikerservaring bij fouten. Het onderscheid tussen een backend-order en een concept-order heeft mij ook geholpen om duidelijker na te denken over wat “permanent” is en wat slechts een noodoplossing is.

10. Globale exception handling voor consistente fout responses (backend)

De backend maakt gebruik van een globale exception handler (@RestControllerAdvice) die verschillende soorten exceptions omzet naar consistente JSON-foutresponses met passende HTTP-statuscodes (bijvoorbeeld 400, 401, 403, 404). Daarnaast wordt in de securityconfiguratie gezorgd dat ongeautoriseerde of ongeldige requests geen standaard HTML-errorpagina's teruggeven, maar ook JSON. Hierdoor ontvangen frontend en API-gebruikers een voorspelbare structuur van foutmeldingen.

Reflectie:

In een vroeg stadium lieten sommige endpoints standaard HTML-errorpagina's van Spring zien, wat in de frontend lastig te verwerken was. Door een centrale exception handler te introduceren, heb ik geleerd hoe belangrijk het is om één uniform fout formaat af te spreken tussen frontend en backend. Dit maakt debugging eenvoudiger en voorkomt dat de frontend voor ieder endpoint aparte foutafhandeling nodig heeft.

11. Domain-level autorisatie met OwnershipGuard en @PreAuthorize (backend)

Naast gewone role-based checks (ROLE_USER/ROLE_ADMIN) heb ik een extra securitylaag geïntroduceerd in de vorm van een OwnershipGuard. Deze component wordt samen met @PreAuthorize gebruikt om te controleren of de ingelogde gebruiker daadwerkelijk eigenaar is van een order of orderregel.

Voorbeelden zijn:

- Alleen de eigenaar (of een admin) mag de factuur van een bepaalde order downloaden.
- Alleen de eigenaar (of een admin) mag het e-book van een bepaalde orderregel downloaden.

Deze checks kijken niet alleen naar rollen, maar ook naar domeinrelaties (bijvoorbeeld: “hoort deze order bij deze gebruiker?”).

Reflectie:

Door met OwnershipGuard en SpEL in @PreAuthorize te werken, heb ik geleerd om security dichterbij het domeinmodel te leggen in plaats van alleen op endpointniveau. Dit is een realistischer patroon uit professionele projecten, waarin niet alleen “rol” maar ook “eigenaarschap” en context bepalen wat een gebruiker mag doen.

Reflectie op het leerproces

Het bouwen van WebBazar heeft mijn begrip van full stack applicatieontwikkeling aanzienlijk verdiept. Ik heb onder andere geleerd om:

- authenticatie en autorisatie strikt te scheiden en op backend-niveau af te dwingen;
- API-contracten duidelijk te definiëren en daar de frontend op aan te sluiten;
- domain modeling toe te passen voor entiteiten als Order, OrderItem, Product en Rental;
- foutafhandeling gestructureerd in te richten, zowel aan frontend- als backend zijde;
- state management in React op een schaalbare manier te organiseren met Context API;
- security niet alleen te zien als “ingelogd of niet”, maar ook als eigenaarschap binnen het domein (via OwnershipGuard).

Wat vooral goed liep, was het ontwerpen en implementeren van een duidelijke service-architectuur in de backend, waarbij business logica niet in controllers maar in services is ondergebracht. Ook ben ik tevreden over de combinatie van role-based security en domain-level checks voor downloads en order inzage.

Verbeterpunten liggen met name op het gebied van testautomatisering en logging/monitoring: er zijn al integratie- en unit-tests aanwezig, maar de dekking is nog niet volledig, en een volwassen logging- en monitoring setup ontbreekt nog. Ook op het vlak van UX (bijvoorbeeld nog duidelijkere foutmeldingen aan de gebruiker) is verdere verfijning mogelijk.

Limitaties en mogelijke doorontwikkeling

1. Geen integratie met een echte betaalmethode

Omdat dit een conceptueel prototype is, wordt de betaling niet daadwerkelijk uitgevoerd en is er geen koppeling met iDEAL, Mollie of vergelijkbare providers.

Doorontwikkeling:

In een volgende versie wil ik een echte betaalprovider integreren (bijvoorbeeld Mollie of Stripe) en de orderstatus direct koppelen aan de uitkomst van de betalingstransactie. Ook zou dan beter moeten worden nagedacht over refund scenario's en het annuleren van orders.

2. Onvolledige testdekking

Er zijn al geautomatiseerde tests aanwezig (zowel integratietests als unit-tests voor o.a. services en security), maar de testdekking is nog niet volledig. Met name randgevallen, foutpaden en sommige delen van de huur- en downloadlogica worden nog voornamelijk handmatig getest.

Doorontwikkeling:

Ik wil de testset uitbreiden met meer JUnit-tests, Mockito-gebaseerde servicetests en eventueel end-to-end tests met bijvoorbeeld Cypress, zodat regressies sneller worden ontdekt en kritieke functionaliteiten automatisch worden bewaakt. Zeker voor security- en download functionaliteit is goede testdekking belangrijk.

3. Alleen lokale bestandsopslag

Bestanden (zoals e-books en facturen) worden momenteel lokaal opgeslagen in een upload-map. Dit is voldoende voor een prototype, maar beperkt schaalbaarheid en betrouwbaarheid (bijvoorbeeld bij server crashes of meerdere servers).

Doorontwikkeling:

In een toekomstig scenario wil ik bestandsopslag verplaatsen naar een cloudoplossing zoals Amazon S3 of Azure Blob Storage, mogelijk in combinatie met een CDN voor snellere downloads en betere beschikbaarheid. Daarbij horen ook zaken als lifecycle-policy's, versiebeheer van bestanden en beveiligde pre-signed URLs.

4. Geen e-mailnotificaties en wachtwoord reset proces

Gebruikers ontvangen op dit moment geen e-mails, bijvoorbeeld bij registratie, bestelling of het naderen van het einde van een huurperiode. Ook is er geen functionaliteit voor wachtwoordherstel via e-mail.

Doorontwikkeling:

In een volgende iteratie zou ik een e-mailservice willen integreren (bijvoorbeeld via SMTP of een service als SendGrid) om orderbevestigingen, facturen en waarschuwingen over huurverval automatisch te versturen. Ook zou ik een token-based wachtwoord reset flow willen toevoegen om het accountbeheer meer in lijn te brengen met realistische webapplicaties.

5. Huurverval nog niet volledig geautomatiseerd en gesignaleerd

De huur functionaliteit wordt in de backend gemodelleerd via de Rental-entiteit met een start- en einddatum, en huurorders worden correct opgeslagen. Bij het downloadendpoint voor e-books wordt huurverval al afgedwongen: als de huur nog niet is ingegaan of al is verlopen, wordt de download geblokkeerd.

Wat echter nog ontbreekt, is aanvullende automatisering en signalering, zoals:

- een achtergrondtaak die verlopen huurperioden periodiek doorloopt en bijvoorbeeld statussen bijwerkt;
- notificaties richting de gebruiker dat een huur bijna afloopt of is verlopen;
- een volledig uitgewerkte UX rond verlopen huur (bijvoorbeeld speciale meldingen in de frontend).

Doorontwikkeling:

In een volgende versie wil ik een backend-taak (bijvoorbeeld via een scheduler) toevoegen die regelmatig controleert op verlopen huurperioden en deze status actief bijwerkt. Ook wil ik het download- en UI-gedeelte uitbreiden met expliciete meldingen rond huurverval, zodat de gebruiker beter begrijpt waarom een download wel of niet beschikbaar is.

Conclusie

De architectuur en implementatie van WebBazar zijn ontworpen om realistische full stack werkwijzen te simuleren binnen een educatieve context. De gemaakte keuzes zijn bewust en weerspiegelen een groeiend inzicht in beveiliging, API-architectuur, domeinmodellering, state management en robuuste foutafhandeling.

Door het combineren van:

1. een gelaagde backend architectuur met DTO's, services en repositories,

Role-based én domain-level autorisatie,

2. een React-frontend met Context API, private routes en een fallback mechanisme bij checkout,

Het ontwikkelen van WebBazar heeft mij geholpen om de samenhang tussen frontend, backend en beveiliging beter te begrijpen. De technische keuzes zijn bewust gemaakt om een veilige, schaalbare en onderhoudbare architectuur te realiseren.

De opgedane kennis rondom JWT-authenticatie, DTO-gebruik en React Context vormt een solide basis voor verdere ontwikkeling en toepassing in realistische en professionele ontwikkeltrajecten.

Bronvermelding

Bij het maken van dit project heb ik uiteraard gebruikgemaakt van de lesstof. Daarnaast heb ik informatie opgezocht op internet. In beperkte mate heb ik ChatGPT geraadpleegd, voornamelijk voor kleine verduidelijkingen en incidentele feedback wanneer bepaalde onderdelen niet direct duidelijk waren. Deze ondersteuning vond uitsluitend plaats voor educatieve doeleinden, en de inhoud en uitwerking van dit project zijn mijn eigen verantwoordelijkheid.

GitHub-links

Volledige applicatie:

<https://github.com/Eyakem1/webbazar-eindopdracht>

Frontend:

<https://github.com/Eyakem1/webbazar-eindopdracht/tree/main/frontend>

Backend:

<https://github.com/Eyakem1/webbazar-eindopdracht/tree/main/backend>